

Trajectory Branching and Tree-Based Credit Assignment for Large Language Models Under Final-Answer-Only Supervision

Executive summary

Final-answer-only reinforcement learning provides an inexpensive, scalable supervisory signal, but it does not reveal which reasoning token, reasoning step, tool call, or intermediate decision caused success or failure. Flat group-based methods such as GRPO therefore assign essentially the same outcome-derived advantage to every token in a trajectory. Tree-based methods improve attribution by generating or discovering **counterfactual continuations from shared prefixes**, evaluating only their terminal answers, and backing those outcomes up to intermediate states. ¹

The literature separates into three families. **Active branching** methods—VinePPO, TreeRPO, TreeRL, Tree-GRPO, and BranPO—deliberately resample suffixes from selected prefixes. They produce the strongest counterfactual signal but add rollout cost. **Retrospective tree construction** methods—TEMPO and RTMC—reuse ordinary rollout groups or logged trajectories, making them computationally attractive but dependent on prefix overlap or a reliable state abstraction. **Search-and-distill** methods such as ReST-MCTS* use terminal answers to train a process reward model and recursively improve both search and the policy, but introduce additional model bias and complexity. ²

The strongest general recommendation is not a single algorithm but a design: use an on-policy tree; branch at uncertain or outcome-contrastive points; estimate conditional success with several suffixes; shrink estimates at low-count nodes; combine local sibling contrasts with a stable prompt-level baseline; and compare methods under equal generated-token, tool-call, and wall-clock budgets. For single-turn verifiable reasoning, VinePPO or entropy-guided TreeRL offers the clearest high-compute path, while TEMPO is the best low-overhead starting point. For long-horizon agents, selective branching such as BranPO or step-level Tree-GRPO is more credible than exhaustive token-level trees. Existing evidence is promising but fragmented across models, tasks, and accounting conventions; no paper yet establishes a compute-matched, causal, cross-domain winner. ³

Background and analytical framework

Why final-answer-only credit assignment is difficult

Let a model generate a trajectory

$$\tau = (a_1, \dots, a_T), \quad s_t = (x, a_{<t}),$$

and let a terminal verifier produce $R(\tau)$, often binary correctness against the reference answer. The optimization target is

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)],$$

with a policy-gradient estimator of the form

$$\nabla_\theta J \approx \sum_{t=1}^T \hat{A}_t \nabla_\theta \log \pi_\theta(a_t | s_t).$$

The difficult quantity is $\hat{A}_t$: how much better was the chosen action than plausible alternatives at that particular prefix? A single completed trajectory supplies $R(\tau)$, but no observation of what would have happened had the model chosen another formula, plan, tool call, or reasoning step.

Under outcome-only GRPO, the terminal rewards of several responses to the same prompt are normalized, after which every token in a response receives the same normalized value. This is a useful variance-reduction baseline but does not distinguish a decisive early insight from routine algebra or an irrelevant tail.

⁴ VinePPO’s diagnostic experiments further show that a conventionally trained PPO critic can have apparently healthy aggregate explained variance while remaining poor at ranking nearby candidate reasoning steps—the ranking capability needed for local credit assignment. ⁵

Final-answer-only attribution faces several distinct statistical problems:

Delayed and sparse observations. For a sequence with thousands of tokens, only the terminal state is directly labeled. Multi-turn agent trajectories can be longer still because observations and tool outputs accumulate in the context. ⁶

Counterfactual non-identifiability. A successful final answer does not prove that every preceding step was helpful, nor does a failed answer establish where failure became inevitable. Without suffix resampling or strong modeling assumptions, individual action effects cannot be identified from a single path.

Suffix-policy confounding. The measured value of an intermediate action is conditional on the policy used to complete the remainder. A good action followed by poor sampling can look bad; a poor action can occasionally be repaired. Tree estimates are therefore policy-conditional rather than intrinsic labels.

Branch-selection bias. Entropy-based, late-stage, random, or failure-triggered branching samples different state distributions. An estimator may be unbiased conditional on a sampled prefix while the training data remain biased toward particular kinds of prefixes.

Degenerate groups. If all sibling leaves succeed or all fail, a group-relative estimator has zero or near-zero variance. TreeRPO prunes such groups, Tree-GRPO combines intra-tree and inter-tree baselines, and BranPO explicitly seeks branches with contrasting outcomes. ⁷

Verifier aliasing. Matching the reference answer establishes outcome correctness under the verifier, not necessarily the validity, faithfulness, or efficiency of the reasoning process. A tree can accurately propagate an imperfect terminal label.

State identity. Exact token prefixes define states naturally in single-turn autoregressive reasoning, but agent environments require a decision about whether two histories are equivalent. RTMC’s software-

engineering experiments, for example, use handcrafted signatures based on file operations, view regions, and content hashes. 8

The common tree estimator

Tree methods approximate conditional values using terminally evaluated suffixes. For a node s with descendant leaves $D(s)$,

$$\hat{V}(s) = \frac{1}{|D(s)|} \sum_{\ell \in D(s)} R_\ell.$$

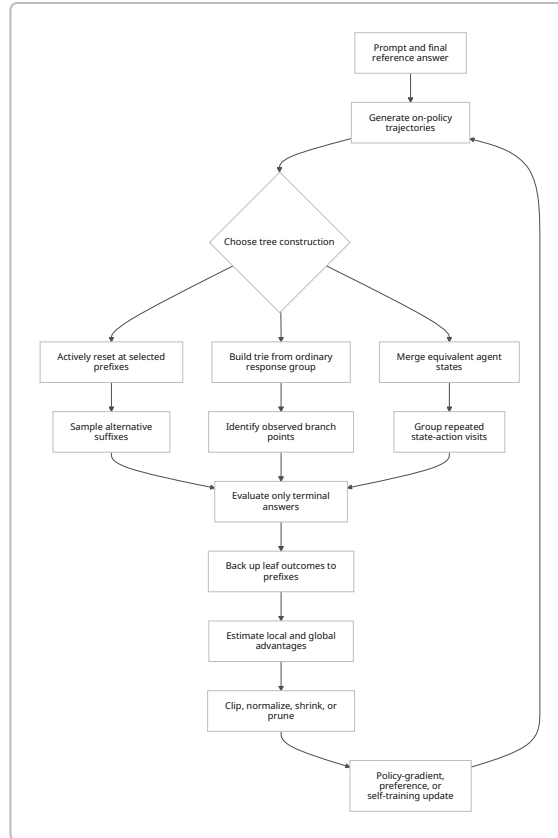
An edge or action can then receive a local temporal-difference-style estimate,

$$\hat{A}_{\text{local}}(s, a) = \hat{V}(s \circ a) - \hat{V}(s),$$

or a sibling-relative estimate,

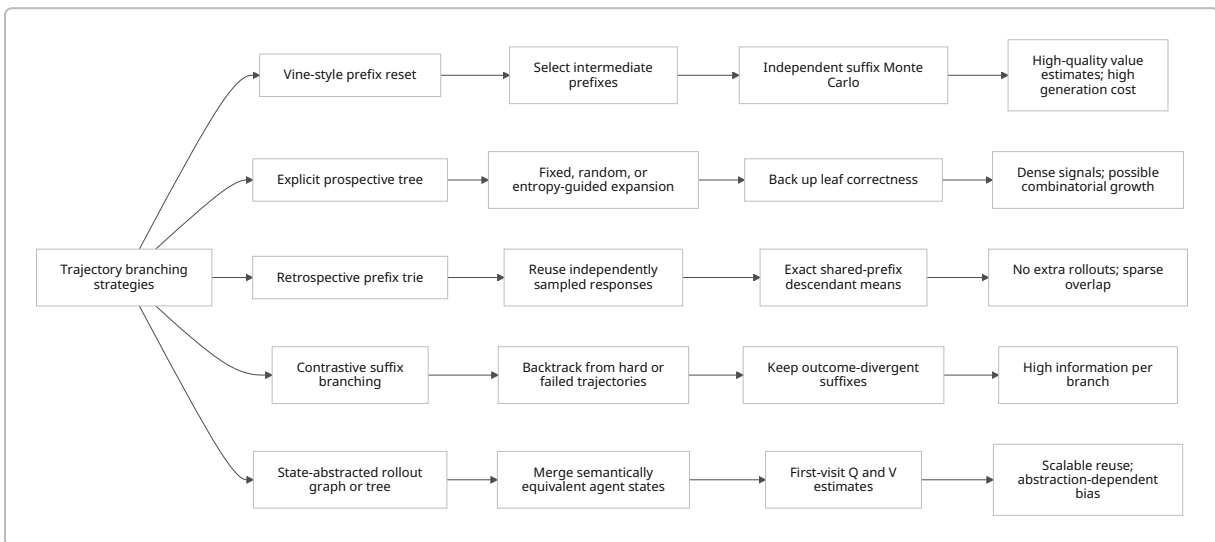
$$\hat{A}_{\text{sibling}}(s, a) = \hat{Q}(s, a) - \frac{1}{|\mathcal{A}_s|} \sum_{a' \in \mathcal{A}_s} \hat{Q}(s, a').$$

These estimates need no manually labeled intermediate solution steps. Their “process supervision” is generated statistically by comparing terminal outcomes below common prefixes. VinePPO uses independent suffix Monte Carlo estimates; TreeRL and TreeRPO back up leaf correctness through explicit trees; TEMPO uses descendant averages in an implicit prefix trie; Tree-GRPO contrasts subtrees at agent-step branches; and RTMC estimates first-visit Monte Carlo Q values after merging equivalent states. 9



How branching strategies differ

The major methodological difference is not merely “tree versus chain,” but **where branching occurs, how alternatives are chosen, and whether additional samples are generated.**



“Tree” is consequently used at several granularities: individual tokens in TEMPO and parts of TreeRL; fixed-length reasoning segments in TreeRPO; complete thought-action-observation cycles in Tree-GRPO;

selected trajectory suffixes in VinePPO and BranPO; and abstract environment states in RTMC. Comparisons that ignore granularity can confound credit-assignment quality with the number and cost of available branch points. ¹⁰

Detailed review of primary papers

VinePPO

Bibliographic record. *VinePPO: Refining Credit Assignment in RL Training of LLMs*, Amirhossein Kazemnejad, Milad Aghajohari, Eva Portelance, Alessandro Sordoni, Siva Reddy, Aaron Courville, and Nicolas Le Roux, 2025, Proceedings of the 42nd International Conference on Machine Learning, PMLR 267, pp. 29557–29590. [PMLR paper](#); [arXiv](#); [official code](#). A publisher DOI is unspecified; the archival arXiv DOI is [10.48550/arXiv.2410.01679](#). ¹¹

Problem setup. VinePPO studies single-turn mathematical reasoning with a reward of one for a verifier-matched final answer and zero otherwise. The authors ask whether PPO underperforms not because fine-grained credit assignment is unnecessary, but because its learned value network gives poor local estimates. They compare PPO’s critic against high-budget Monte Carlo estimates and find that it often struggles to rank alternative intermediate states even when aggregate critic diagnostics appear healthy. ⁵

Model, branching, and estimator. A main response is sampled from the current policy. At selected intermediate prefixes, the environment is “reset” simply by placing that prefix back in the model context, and K independent continuations are generated. Their terminal correctness estimates

$$\hat{V}_{\text{MC}}(s_t) = \frac{1}{K} \sum_{k=1}^K R(\tau^{(k)} \mid s_t).$$

The step advantage is based on the change in conditional success probability between adjacent states, approximately $\hat{V}(s_{t+1}) - \hat{V}(s_t)$. The structure resembles a vine: one main trajectory with local bundles of suffix branches, rather than an exhaustively expanded tree. ¹²

Loss and training. VinePPO preserves PPO’s clipped policy objective and KL regularization but replaces the learned critic and generalized advantage estimation with Monte Carlo state values. The PPO-style objective uses token-level importance ratios, clipping, the Monte Carlo-derived advantages, and a KL penalty to the reference model; no value-network loss is required. Rewards occur only at the final token, with $\gamma = 1$ in the reported experiments. ¹³ The policy is trained on fresh on-policy responses, while suffix rollouts are used to improve the advantage estimate. The authors ablate $K \in \{1, 3, 9\}$ and report that larger K reduces estimator variance sufficiently to offset slower iterations. ¹⁴

Datasets, models, and metrics. Experiments use MATH with the OpenAI/PRM split—11,500 training, 500 validation, and 500 test problems—and GSM8K with 7,100 training, 373 validation, and 1,319 test examples. Models include RhoMath-1.1B and DeepSeekMath-7B. Evaluation covers final-answer accuracy, gradient steps, wall-clock time, KL divergence, generalization slope, value-estimation mean absolute error and explained variance, and action-ranking accuracy. ¹⁵

Key results. VinePPO reaches higher final test accuracy than PPO and critic-free baselines in the reported math settings. Relative to PPO, it reaches PPO’s peak accuracy in approximately nine times fewer gradient steps and three times less wall-clock time for RhoMath-1.1B, and about 2.8 times fewer steps and 1.51 times less time for DeepSeekMath-7B, despite each VinePPO iteration being more expensive. ¹⁴

Strengths and limitations. Its primary strength is conceptual cleanliness: it directly estimates the policy-conditional probability of eventual correctness and isolates the value-estimation variable while retaining the rest of PPO. It is especially compelling when prefixes can be restored cheaply, the terminal verifier is reliable, and compute is available. Its main limitation is suffix-generation cost, which grows with the number of evaluated prefixes and continuations. It also estimates values only under the current suffix policy; estimates can be noisy when success is rare, and the experiments are concentrated on two mathematical datasets and models no larger than 7B. The method does not solve semantic state equivalence or stochastic external-environment reset.

Reproducibility. Code is public, and the paper provides dataset splits, hyperparameters, verifier details, and extensive value-estimation diagnostics. Reproducibility is **high** relative to the field. ¹⁶

ReST-MCTS*

Bibliographic record. *ReST-MCTS: LLM Self-Training via Process Reward Guided Tree Search**, Dan Zhang, Sining Zhoubian, Ziniu Hu, Yisong Yue, Yuxiao Dong, and Jie Tang, 2024, Advances in Neural Information Processing Systems 37. [NeurIPS paper](#); [arXiv](#); [official code](#). A proceedings DOI is unspecified; the archival DOI is [10.48550/arXiv.2406.03816](https://doi.org/10.48550/arXiv.2406.03816). ¹⁷

Problem setup. ReST-MCTS* assumes questions with known oracle final answers but no annotated intermediate steps. Its goal is to turn terminal correctness into a process reward model and to use that model to find better reasoning traces for self-training. It is therefore not a pure online policy-gradient algorithm; it is an iterative search, labeling, and supervised self-training system. ¹⁸

Model and tree construction. Each MCTS node represents a partial reasoning solution. Search comprises node selection, thought expansion, Monte Carlo completion, and value backpropagation. Terminal rollouts are checked against the oracle answer, and the empirical probability that a partial solution leads to a correct answer becomes a target for the process reward model. The learned PRM subsequently guides tree selection and expansion, generating increasingly strong candidate paths. ¹⁹

Objectives. The process reward model is trained with a mean-squared-error objective against rollout-derived values for question–partial-solution pairs. The policy is then supervised on positively selected complete reasoning trajectories using next-token likelihood. Thus, the terminal answer is the primitive label, but an auxiliary PRM converts it into learned process supervision before policy training. ²⁰

Training procedure. The authors initialize a policy and PRM, run guided tree search, label nodes from terminal rollouts, train the PRM on generated partial traces, fine-tune the policy on high-quality positive trajectories, and repeat. This creates a bootstrapping loop: improved search yields better PRM data and policy demonstrations, while the improved models support stronger subsequent search. ¹⁸

Datasets, tasks, and metrics. The system is trained or evaluated on GSM8K, MATH, TheoremQA, SciBench, and SciEval, using several open and proprietary language-model backbones. Reported metrics include exact-answer accuracy, verifier accuracy, and search performance under token or prompt budgets. ¹⁹

Key results. In the reported self-training comparisons, later ReST-MCTS *iterations improve average mathematical performance over ReST-EM and self-rewarding baselines. Its verifier reaches 87.5% on GSM8K and 39.0% on MATH500 in one Mistral/MetaMath setting. On SciBench, search guided by ReST-MCTS* also improves over chain-of-thought and Tree-of-Thought prompting across GLM-4, GPT-3.5, and Llama-2 backbones, though absolute accuracy remains low on the hardest scientific problems. ¹⁹

Strengths and limitations. ReST-MCTS* demonstrates that final answers can generate process labels without human step annotation and that search and training can improve one another. It is useful when one wants a reusable PRM or high-quality synthetic reasoning corpus rather than a strictly on-policy estimator. The limitations are substantial: MCTS and repeated completion are expensive; PRM errors can become self-reinforcing; selected traces induce distribution shift; and the system contains more moving parts than critic-free tree estimators. The authors also observe rapid prompt-token growth with branching and little gain on tasks where search is unnecessary or alternatives are easy to distinguish. ²¹

Reproducibility. Official code is public and the paper describes its iterative components and evaluations. Full reproduction still requires careful reconstruction of model-serving and search budgets. Reproducibility is **medium-high**. ¹⁸

TreeRL

Bibliographic record. *TreeRL: LLM Reinforcement Learning with On-Policy Tree Search*, Zhenyu Hou, Ziniu Hu, Yujia Li, Rui Lu, Jie Tang, and Yuxiao Dong, 2025, ACL 2025 Long Papers, pp. 12355–12369. [ACL Anthology](#); [DOI 10.18653/v1/2025.acl-long.604](#); [official code](#). ²²

Problem setup. TreeRL addresses on-policy RL for math and code reasoning with terminal answer or test-case correctness. It seeks both better exploration and denser process credit without an externally trained PRM. ²³

Tree construction. Its EPTree sampler begins with M complete trajectories, computes token entropy, selects the top- N uncertain positions across existing trees, and generates T continuations from each selected fork. This expansion is repeated for L rounds, producing $M(1 + NLT)$ leaves. Because many forks can be generated in parallel and the paper typically uses roughly two expansion rounds, EPTree is more serving-friendly than sequential MCTS. ²⁴

Value and credit estimator. A node value is the fraction of correct descendant leaves,

$$V(s) = \frac{1}{|L(s)|} \sum_{\ell \in L(s)} \mathbf{1}[\ell \text{ correct}].$$

TreeRL combines a global advantage $V(s) - V(\text{root})$ and a local advantage $V(s) - V(\text{parent}(s))$. Because shared prefixes appear in several extracted leaf sequences, the resulting signal is divided by $\sqrt{|L(s)|}$ to reduce repeated-prefix over-weighting. ²⁵

Loss and training. The paper’s base RL objective is a policy-gradient likelihood objective weighted by an advantage. TreeRL substitutes the tree-derived process signal for the ordinary sequence-level advantage and updates the policy on sequences extracted from fresh on-policy trees. The paper does not introduce a separately named clipped surrogate analogous to TreeRPO; exact optimizer-level clipping and regularization choices are implementation details rather than the main formal contribution and should be treated as unspecified unless taken from the released configuration. ²⁶

Datasets, models, and metrics. The principal models are GLM4-9B and Qwen2.5-14B initialized through supervised fine-tuning. Evaluation includes MATH500, an Omni-MATH 500-example subset, AIME 2024, AMC, OlympiadBench, and LiveCodeBench. Metrics are final-answer or code-test accuracy, average accuracy, PassRate under generation-token budgets, and wall-clock/search efficiency. ²⁷

Key results. TreeRL improves over ChainRL on both principal model families; the headline comparison reports approximately 2.1 percentage points over ChainRL for GLM4-9B and 2.9 points for Qwen2.5-14B. An ablation with 30 responses reports 81.7 on MATH500, 36.7 on the Omni-MATH subset, 28.0 on AIME, 55.9 on AMC, 44.6 on OlympiadBench, and 20.8 on LiveCodeBench for the full global-plus-local, subtree-normalized signal. Removing individual components reduces average performance. ²⁸

Strengths and limitations. Entropy-guided expansion spends branches where the policy is uncertain, and local-plus-global credit combines fine discrimination with a stable prompt-level reference. It is fully on-policy and requires neither a critic nor a separately trained PRM. However, uncertainty is not identical to decision importance: high entropy can reflect stylistic alternatives, while confident errors can be missed. The tree implementation is reported to be roughly twice as slow as multi-chain sampling because current inference engines lack specialized tree optimizations. The estimator also gives more reliable values to large subtrees than to lightly sampled nodes, despite the heuristic normalization. ²⁹

Reproducibility. Official code, formulas, task lists, and ablations are available. Reproducibility is **high**, although exact systems throughput depends on tree-aware serving support. ³⁰

TreeRPO

Bibliographic record. *TreeRPO: Tree Relative Policy Optimization*, Zhicheng Yang, Zhijiang Guo, Yinya Huang, Yongxin Wang, Yiwei Wang, Xiaodan Liang, and Jing Tang, 2025, arXiv preprint. [Paper](#); [archival DOI 10.48550/arXiv.2506.05183](#); [linked repository](#). A peer-reviewed venue is unspecified as of August 4, 2026. The repository name does not match the paper title, which makes artifact discovery less transparent. ³¹

Problem setup. TreeRPO targets verifiable mathematical reasoning using only terminal correctness. It argues that a group of independent complete responses gives a weak, trajectory-level contrast, whereas alternatives sampled from the same intermediate state provide a more relevant local baseline. ³²

Tree construction. The method builds an explicit N -ary reasoning tree to depth D . At each node, the policy samples N continuations of a fixed token length L_{step} ; each continuation forms a child and expansion proceeds recursively. Leaves are completed to final answers and assigned verifier rewards. Parent values are obtained by averaging child values upward through the tree. ³²

Objective. Children sharing a parent form a local comparison group. Rewards or backed-up values are normalized within the sibling group to obtain relative step advantages. Groups with uniform outcomes are

pruned because they provide no relative signal. TreeRPO uses a GRPO-style clipped importance-ratio objective and a KL penalty, but replaces flat prompt-level groups with tree-local sibling groups. ³³

Training procedure. New trees are generated on-policy; terminal final answers are verified; values are propagated upward; informative sibling groups are extracted; and the policy is optimized on their token spans. The fixed-length segments operationalize “reasoning steps” without requiring a semantic step parser. ³⁴

Datasets, models, and metrics. Training uses roughly 7,500 MATH examples, with evaluation on MATH500, OlympiadBench, Minerva Math, and AIME 2024. The principal reported model is Qwen2.5-Math-1.5B. Metrics include Pass@1 accuracy, average benchmark performance, response length, and ablations over tree parameters and group construction. ³⁵

Key results. The paper reports an increase in average Pass@1 from 19.0 for the starting model to 35.5 after TreeRPO training, a 2.9-point gain over its GRPO comparison, together with responses that are approximately 18.1% shorter. ³²

Strengths and limitations. TreeRPO has a particularly direct interpretation: local sibling outcomes approximate action-level counterfactuals, and the GRPO machinery can be reused almost unchanged. Its fixed segmentation avoids an auxiliary step model. The principal limitation is combinatorial expansion: naive cost scales approximately as N^D , making deep trees impractical without pruning, selective expansion, or prefix-cached serving. Fixed token windows can also split coherent reasoning operations or combine several decisions into one node. Evidence is limited to a 1.5B model, and the authors explicitly cite resource constraints. ³⁶

Reproducibility. A public repository is linked and paper details are available, but the artifact naming mismatch and restricted model scale lower confidence compared with VinePPO or TreeRL. Reproducibility is **medium**. ³⁷

TEMPO and Prefix-to-Tree

Bibliographic record. *Exploiting Tree Structure for Credit Assignment in RL Training of LLMs*, Hieu Tran, Zonghai Yao, and Hong Yu, 2026, Findings of ACL 2026. The tree-construction procedure is Prefix-to-Tree, and the optimization algorithm is TEMPO—Tree-Estimated Mean Prefix Value for Policy Optimization. [ACL paper](#); [arXiv](#); [DOI 10.18653/v1/2026.findings-acl.524](#); [official code](#). ³⁸

Problem setup. TEMPO assumes a standard group of multiple responses sampled for each prompt and a binary verifier that checks only the final answer. Unlike active branching methods, it does not request extra continuations or alter the sampler. It asks whether the implicit prefix tree already present in a GRPO response group can recover token-level information. ⁴

Tree construction. Prefix-to-Tree inserts each sampled token sequence into a trie. Every exact prefix is a node, and divergence between responses creates branches. A prefix value is the mean terminal reward of all completions sharing it:

$$V(s_t) = \frac{1}{|D(s_t)|} \sum_{j \in D(s_t)} r_j.$$

Along a unique, nonbranching continuation, the descendant set is unchanged, so $V(s_{t+1}) = V(s_t)$. A nonzero temporal-difference term appears only when extending through a branch that changes the descendant outcome distribution. ⁴

Objective. TEMPO combines the usual group-relative Monte Carlo term with a branch-local correction:

$$\hat{A}_{i,t} = \frac{r_i - \text{mean}(r) + V(s_{t+1}) - V(s_t)}{\text{std}(r)}.$$

The policy update uses a DAPO-style token-level clipped objective with asymmetric “clip-higher,” global token averaging, and no KL term in the reported formulation. At nonbranching tokens, the TD correction is zero and the update reduces to the underlying GRPO-like signal. ⁴

Training procedure. Six responses are sampled per question, and the authors run ten passes through each training dataset, giving 60 episodes per question for all compared methods. PPO, GRPO, HEPO, and TEMPO consume the same number of episodes; TEMPO adds only trie construction and value aggregation. ⁴

Datasets, models, and metrics. Training uses MATH and MedQA. Out-of-distribution evaluations use GSM-HARD and AMC23 for mathematics, and MedMCQA and the medical portion of MMLU for medicine. Models are Qwen3-1.7B and Qwen3-4B. Metrics include benchmark accuracy, validation convergence, wall-clock time, group-size effects, branch count and location, and out-of-distribution generalization. ⁴

Key results. On Qwen3-1.7B, TEMPO reports 87.0 on MATH versus 82.4 for GRPO and 59.15 on MedQA versus 56.24 for GRPO. On Qwen3-4B, it reports 91.0 versus 87.6 on MATH and 79.18 versus 76.12 on MedQA. TEMPO also leads the listed baselines on all four reported out-of-distribution benchmarks while retaining comparable wall-clock cost. ⁴

Strengths and limitations. TEMPO is the lowest-friction method in the review: no extra model, judge, process labels, or rollout schedule is required. It is a plausible first modification to a GRPO implementation. Its limitation is that its tree is observational rather than experimentally designed. Independent responses often diverge early, leaving many deeper prefixes with one descendant and no local contrast. Exact token identity also treats semantically equivalent prefixes as different states. Thus, nominally token-level credit may be concentrated at only a few early divergence points, and the quality of the tree depends strongly on group size and decoding diversity.

Reproducibility. Code is public, the objective is explicit, and compute is matched by episode count. Reproducibility is **high**, subject to reproducing the exact verifier and Qwen training stack. ³⁹

Tree-GRPO

Bibliographic record. *Tree Search for LLM Agent Reinforcement Learning*, Yuxiang Ji, Ziyu Ma, Yong Wang, Guanhua Chen, Xiangxiang Chu, and Liaoni Wu, 2026, ICLR 2026; the method is Tree-based Group Relative Policy Optimization, or Tree-GRPO. [OpenReview](#); [arXiv](#); [archival DOI 10.48550/arXiv.2509.21240](#); [official code](#). A separate conference DOI is unspecified. ⁴⁰

Problem setup. Tree-GRPO addresses long-horizon ReAct agents whose contexts contain repeated thought-action-observation cycles and whose only training reward is a terminal outcome score. Each tree node is a complete agent step rather than a token or sentence, giving a semantically meaningful branch granularity for tool-using trajectories. ⁴¹

Tree construction. For each prompt, the method initializes M independent full trajectories. It then randomly selects N nonterminal nodes in each tree and resumes the agent from each selected history to create new suffixes. Sampling and expansion are repeated L times. The final group contains $M(LN + 1)$ rollouts. Because a resumed branch reuses the prefix, its expected incremental token and tool-call cost is lower than a new complete chain; however, concentrating many branches in few trees narrows global exploration. ⁴¹

Credit estimator and objective. Outcomes are normalized within two groups. The **intra-tree advantage** compares a trajectory against branches sharing a tree and therefore provides a local, prefix-conditioned preference. The **inter-tree advantage** normalizes across all trees for the prompt and supplies a more stable global baseline. Their sum weights a GRPO-style clipped token-level importance-ratio objective with KL regularization:

$$\hat{A}_{\text{tree}} = \hat{A}_{\text{intra}} + \hat{A}_{\text{inter}}.$$

The authors prove that, under a binary winner/loser assumption, the gradient structure of intra-tree GRPO is equivalent to step-level direct preference optimization up to a weighting term. ⁴²

Training procedure, tasks, and metrics. The policy repeatedly collects fresh agent trees, receives terminal rewards, computes both advantage levels, and performs on-policy updates. The evaluation spans 11 datasets: NQ, TriviaQA, and PopQA for single-hop retrieval; HotpotQA, 2WikiMultiHopQA, MuSiQue, and Bamboogle for multi-hop retrieval; and SimpleQA, GAIA, WebWalkerQA, and BrowseComp for harder web-agent evaluation. Models range from Qwen2.5-1.5B to 14B and include Llama-3.2-3B-Instruct. Metrics are task accuracy and performance under token or tool-call rollout budgets. ⁴³

Key results. For Qwen2.5-3B, the repository reports a multi-hop average of 36.8 for Tree-GRPO versus 31.8 for chain GRPO. For Qwen2.5-14B, the corresponding averages are 45.3 and 41.8. The paper reports that its 3B configuration can surpass chain-based training while using approximately one quarter of the rollout budget. An ablation shows that, at a budget of roughly four rollouts per prompt, intra-tree normalization alone collapses, inter-tree alone reaches 33.8 average, and their combination reaches 36.8 versus 31.8 for chain sampling. ⁴⁴

Strengths and limitations. Agent-step nodes avoid arbitrary token segmentation, and prefix reuse directly reduces repeated tool calls. The intra/inter decomposition addresses the high variance of small sibling groups. Random expansion is simple and avoids a learned selector, but it may spend branches on low-impact states. The theoretical preference equivalence assumes binary outcomes and does not establish unbiased causal attribution in general. Tool environments must also be replayable or resumable, and sequential interaction remains a systems bottleneck.

Reproducibility. Code, training scripts, retrieval setup, and preliminary logs are public. The repository initially described parts of the web-agent data as forthcoming, so reproducibility is **high for the released retrieval tasks and medium for the full 11-dataset claim**. ⁴⁵

BranPO

Bibliographic record. *BranPO: Scalable Contrastive Branch Sampling for Long-Horizon Agentic Reinforcement Learning*, Yubao Zhao, WeiQuan Huang, Sudong Wang, Ruochen Zhao, Chen Chen, Yao Shu, and Chengwei Qin, 2026, arXiv preprint. [Paper](#); [archival DOI 10.48550/arXiv.2602.03719](#); [official code](#). A peer-reviewed venue is unspecified. The current title supersedes the earlier preprint title *Training Multi-Turn Search Agent via Contrastive Dynamic Branch Sampling*. ⁴⁶

Problem setup. BranPO focuses on long-horizon multi-turn search agents. Flat GRPO rewards or penalizes every step in a completed episode equally, while exhaustive tree construction is expensive. BranPO seeks a small number of high-information counterfactual suffixes whose terminal outcomes disagree. ⁴⁷

Branch construction. Given a completed trajectory, the method truncates it at an intermediate prefix and samples alternative suffixes. A suffix is retained only when its terminal outcome contrasts with the original—successful versus failed. Difficulty-aware sampling changes the search budget: easier successful trajectories receive limited late branching, whereas difficult or failed trajectories trigger recursive backtracking toward earlier steps until a contrastive branch is found or the budget is exhausted. ⁴⁸

Objective. BranPO uses hierarchical group-relative advantages. A prefix-level advantage compares the average conditional success of branches rooted at different prefixes, while a branch-level advantage contrasts successful and failed suffixes sharing the same prefix. Tokens before the branch receive the prefix signal; suffix tokens receive the branch-conditioned signal. The policy objective retains a GRPO-like KL-regularized policy gradient. The authors show that the formulation reduces to GRPO under a single-branch special case and relate the suffix update to preference learning. A redundant-step mask suppresses tail steps judged not to contribute additional information. ⁴⁹

Training procedure. Experiments begin with supervised cold-start data of approximately 10,000 correct trajectories, followed by fully on-policy RL using rLLM and veRL. Training first permits shorter interaction horizons and then extends to longer horizons. Branches are generated and filtered online using terminal outcome contrast. ⁵⁰

Datasets, models, and metrics. Training uses Search-R1 and ASearcher-style search-agent data. Evaluation covers HotpotQA, 2WikiMultiHopQA, MuSiQue, Bamboogle, GAIA, and ALFWorld, with horizons as long as 50 environment steps. Metrics include exact match, token-level F1, LLM-as-judge correctness, Avg@4, and environment success rate. ⁵¹

Key results. On the Search-R1 multi-hop comparison, BranPO reports an average exact-match/F1 of 42.2/53.2, compared with 38.9/48.7 for AEPO and 37.7/48.1 for Tree-GRPO. In ASearcher-based experiments, the reported average F1/LLM-judge score is 55.6/65.2 versus 52.8/62.1 for GRPO. On ALFWorld, BranPO reaches 93.6% success versus 90.8% for GiGPO and 77.6% for GRPO. Removing dynamic branch sampling or the redundant-step mask lowers average performance in the reported ablations. ⁵¹

Strengths and limitations. Outcome-divergent suffixes have high information value: they directly reveal that the shared prefix admits materially different futures. Adaptive backtracking is more scalable than uniform branching over every agent step and is especially appropriate when trajectories are tens of turns long. However, seeking contrastive outcomes changes the sampling distribution and can bias gradient estimates unless treated as a designed preference objective rather than an unbiased on-policy estimator.

Branch search can still become expensive when successes are extremely rare or nearly universal, and the redundant-step mask introduces an additional heuristic. Most evaluations involve search or embodied-agent tasks rather than pure reference-answer reasoning.

Reproducibility. Code is public and the paper documents its stack, cold-start stage, branching components, and ablations. Reproducibility is **medium-high**, with remaining sensitivity to search infrastructure and judge configuration. ⁵²

RTMC

Bibliographic record. *RTMC: Step-Level Credit Assignment via Rollout Trees*, Tao Wang, Suhang Zheng, and Xiaoxiao Xu, 2026, arXiv preprint. [Paper](#); [archival DOI 10.48550/arXiv.2604.11037](#). A peer-reviewed venue and official code repository are unspecified as of August 4, 2026. ⁵³

Problem setup. RTMC considers multi-step agents for which a batch already contains several complete rollouts of the same task. Rather than generating explicit counterfactual branches, it attempts to identify repeated or equivalent state-action visits across those trajectories and estimate local returns from them. Its principal experiment is repository-level software engineering. ⁵⁴

Tree or graph construction. Raw histories are mapped to state-action signatures. Trajectories sharing a signature are merged into a rollout tree or directed acyclic structure. For software engineering, signatures encode operations on files, coarse viewing regions, and content hashes. This semantic abstraction enables trajectories with different text histories but equivalent repository states to contribute to the same estimate. ⁸

Estimator and loss. RTMC computes first-visit Monte Carlo action values,

$$\hat{Q}(s, a) = \frac{1}{N(s, a)} \sum_{i: (s, a) \in \tau_i} G_i,$$

then defines

$$\hat{V}(s) = \frac{1}{|\mathcal{A}_s|} \sum_{a \in \mathcal{A}_s} \hat{Q}(s, a), \quad \hat{A}(s, a) = \hat{Q}(s, a) - \hat{V}(s).$$

The action-level advantage is broadcast across the token span that produced the agent action and used in a GRPO-compatible policy update. A prior or shrinkage term is used at singleton and low-count nodes to reduce variance. Computation is linear in the total number of rollout steps after signatures are available and requires no additional inference model. ⁵⁵

Important supervision caveat. The core estimator can operate on terminal returns, but the main reported software-engineering experiments include small step rewards for valid actions, penalties for invalid actions, and validation-related shaping in addition to the terminal pass/fail reward. RTMC should therefore not be interpreted as having been empirically validated in a strictly final-reference-answer-only regime, even though its mathematical construction is compatible with one. ⁵⁶

Training, datasets, and metrics. The reported policy is Qwen3-Coder-30B-A3B-Instruct trained using ROLL on 32 GPUs. Training uses approximately 8,100 R2E-Gym tasks, and evaluation uses the 500-instance SWE-bench Verified benchmark. The primary metric is Pass@1, aggregated across three runs. ⁵⁷

Key results. The paper reports 52.2% SWE-bench Verified resolution for RTMC, compared with 49.0% for GRPO, 50.4% for GRPO with the same step rewards, and 46.8% for the starting baseline. Removing the low-count prior reduces performance to 49.7%, indicating that naive empirical tree values are too noisy at the available rollout count. ⁵⁸

Strengths and limitations. RTMC extracts more value from trajectories already being collected and is the only reviewed method to confront semantic state merging directly. It is promising for expensive environments where additional tool rollouts dominate cost. Its weakness is that state abstraction becomes part of the learning algorithm: overly specific signatures produce singleton nodes and no contrast, while overly coarse signatures combine meaningfully different states and introduce bias. The paper reports that many nodes remain singletons with eight rollouts, limiting the amount of truly local evidence. The best result also depends on domain-specific reward shaping. ⁵⁹

Reproducibility. Public benchmark data exist, but no official implementation was identified, and the handcrafted signature machinery is central to the result. Reproducibility is **low-to-medium**.

Cross-method comparison

The ratings below are analytical rather than directly comparable leaderboard measurements. “Sample efficiency” distinguishes useful credit per completed trajectory from raw wall-clock efficiency; “scalability” includes both algorithmic growth and compatibility with parallel LLM serving.

Method	Branching strategy	Supervision type	Scalability	Sample efficiency	Principal assumptions	Best-use cases
VinePPO ⁵	Actively resample K suffixes from selected prefixes along a main trajectory	Strict terminal verifier; no learned critic	Medium. Cost grows with evaluated prefixes × suffixes; highly parallelizable	High gradient efficiency; medium rollout efficiency	Prefix can be restored exactly; terminal reward is reliable; sufficient success probability for MC estimation	High-compute single-turn math, code, and other deterministic verifiable reasoning

Method	Branching strategy	Supervision type	Scalability	Sample efficiency	Principal assumptions	Best-use cases
ReST-MCTS* 60	PRM-guided MCTS over partial reasoning traces	Terminal answers converted into learned process-value targets	Low-medium. Search, PRM training, and iterative self-training are expensive	Medium. Search targets promising paths but can consume many completions	PRM generalizes across partial solutions; iterative bootstrapping does not amplify errors	Synthetic reasoning-data generation, reusable verifiers, offline/self-training pipelines
TreeRL 61	Fork at high-entropy token positions; several parallel expansion rounds	Terminal correctness backed up to global and local node advantages	Medium. Parallelizable, but reported tree generation remains about 2× slower than multi-chain	High when entropy predicts decision importance	Entropy identifies useful branch points; exact prefix reset is valid	Math and code RL where exploration and fine credit are both important
TreeRPO 62	Fixed-length, fixed-factor N -ary expansion to depth D	Strict terminal correctness; sibling-relative process advantages	Low-medium. Naive N^D growth	Medium. Strong local contrasts, but many redundant branches	Fixed token spans approximate coherent steps; sibling groups contain reward variation	Controlled small-model studies; shallow trees; research on local relative objectives
TEMPO/P2T 4	Retrospective exact-prefix trie over ordinary response groups	Strict binary final-answer verifier	High. No extra rollout or auxiliary model	High compute efficiency; variable statistical efficiency	Independently sampled answers share useful prefixes; exact token prefix is a meaningful state	Drop-in GRPO improvement under limited inference budget

Method	Branching strategy	Supervision type	Scalability	Sample efficiency	Principal assumptions	Best-use cases
Tree-GRPO 63	Initialize agent chains, then randomly expand complete agent-step nodes	Terminal outcome reward; intra-tree plus inter-tree normalization	Medium-high for tool budgets; medium systems throughput	High under expensive prefixes/tool calls	Environment histories are resumable; agent steps are meaningful action units	Multi-hop retrieval and web agents with costly shared prefixes
BranPO 64	Backtrack and resample suffixes; retain outcome-contrastive branches	Terminal outcomes plus hierarchical branch-relative advantages	Medium-high. Adaptive rather than exhaustive	High when contrasts can be found cheaply	Contrasting suffixes are discoverable; branch-selection bias is acceptable or corrected	Long-horizon search and embodied agents; rare decisive failures
RTMC 65	Retrospective merging by state-action signatures	Terminal return in principle; reported results also use step shaping	High computation after rollouts; domain engineering can be costly	High only when states overlap; low at singleton nodes	State abstraction preserves decision equivalence; repeated visits occur	Expensive software or tool environments with reusable logs and structured states

Comparative interpretation

The most statistically direct methods are VinePPO and explicit local branching methods such as TreeRPO: they intervene on a prefix and observe several alternative futures. The most computationally economical is TEMPO, but it can only exploit branches that happened to occur in the ordinary sample group. TreeRL lies between these extremes, actively choosing uncertain prefixes while limiting expansion rounds. 66

For agent trajectories, token-level trees are usually the wrong abstraction. Tree-GRPO's complete agent-step node and BranPO's suffix branch correspond more closely to actionable decisions, while RTMC goes further by merging environment-equivalent states. This shift improves semantic relevance but introduces environment replay and state-abstraction requirements absent from plain mathematical reasoning. 67

The estimators also differ in bias and variance. Mean descendant reward is an unbiased Monte Carlo estimate for the conditional value of a fixed on-policy prefix under independent continuation sampling, but it can have extreme variance with few leaves. Sibling normalization reduces shared-prefix variance but becomes unstable when siblings have uniform rewards. Selective retention of contrasting outcomes improves information density but changes the sampling distribution. Learned PRMs lower online rollout

requirements after training but add approximation and distribution-shift error. These are trade-offs rather than universally ordered improvements. ⁶⁸

Gaps, open problems, and practical recommendations

Major research gaps

There is no standard credit-assignment benchmark. Papers use different base models, prompts, supervised warm starts, verifiers, rollout counts, response lengths, hardware, and task families. Some match completed episodes, others match generated tokens, and agent papers may match tool calls. These accounting choices can reverse conclusions because a branched suffix is much cheaper than a full trajectory but not free. VinePPO carefully reports wall-clock and gradient efficiency, while Tree-GRPO emphasizes token/tool budgets and TEMPO matches episode counts; these are complementary rather than directly equivalent controls. ⁶⁹

Credit accuracy is seldom evaluated causally. Most evidence is downstream final accuracy. A process advantage could improve optimization while still failing to identify the truly decisive step. VinePPO's comparison against 256-sample Monte Carlo values is a valuable exception, but even that oracle is policy-conditional. A stronger test would intervene on the predicted high-credit action while preserving the surrounding prefix and measure the resulting change in terminal success. ¹⁴

Branch allocation is largely heuristic. Current choices include uniform depth, random nodes, token entropy, late-stage backtracking, and outcome contrast. Entropy can oversample inconsequential lexical uncertainty; random expansion wastes budget; late branching may occur after an irreversible error; and contrastive search may overrepresent borderline trajectories. A learned allocation policy could optimize expected reduction in advantage uncertainty per token or tool call, but it would itself need correction for adaptive sampling.

Low-count value estimates are under-regularized. A mean over one or two leaves is a poor estimate, yet it can receive the same optimization weight as a value supported by dozens of descendants. TreeRL uses a subtree-count heuristic, Tree-GRPO adds an inter-tree fallback, and RTMC uses a prior, but there is no agreed Bayesian or confidence-aware estimator. ⁷⁰

Exact prefixes do not capture semantic state equivalence. TEMPO's trie is safe but sparse; RTMC's abstraction creates more sharing but can merge nonequivalent states. Learning invariant state embeddings for tree merging is attractive, but using a learned representation risks leakage, nonstationarity, and opaque attribution. ⁷¹

Verifier correctness and reasoning correctness are conflated. A final-answer verifier can reward lucky guessing, invalid derivations that end correctly, or exploitation of answer-format weaknesses. Tree backup multiplies this issue by treating the terminal label as evidence about many intermediate nodes. Robust training needs adversarial verifier tests, format controls, and a distinction between outcome accuracy and process validity.

On-policy status is not binary. A suffix sampled from a prefix under the current policy is on-policy conditional on that prefix, but prefix selection may be adaptive, the tree may be reused over several

optimizer steps, and branches may be generated by stale workers. Importance weighting corrects token-policy mismatch but does not automatically correct branch-selection probabilities.

Theoretical results are narrow. Tree-GRPO establishes a structural connection to step-level DPO under binary preference assumptions, while most other algorithms are motivated by Monte Carlo value estimation. Missing results include finite-sample error bounds under adaptive branching, convergence with pruned or contrastively retained branches, bias from state merging, and guarantees for nonstationary tool environments. ⁶³

Large-model evidence remains limited. TreeRPO is evaluated primarily at 1.5B; TEMPO at 1.7B and 4B; VinePPO up to 7B; TreeRL up to 14B; and RTMC uses a 30B-A3B mixture-of-experts coder but with domain-specific shaping. It is unresolved whether stronger base models need fewer branches because their values are easier to estimate, or more branches because their trajectories are longer and more diverse. ⁷²

Practical recommendations

For a standard GRPO math or code pipeline, begin with TEMPO as the minimum-cost intervention. It requires no additional model and preserves the existing sample budget. Instrument the effective number of branch nodes, descendants per node, and fraction of tokens receiving nonzero TD corrections; otherwise an implementation can appear tree-based while supplying little additional information. ⁴

When compute is unconstrained and the verifier is exact, use VinePPO-style suffix Monte Carlo as the reference method. It provides the cleanest estimate of conditional success and can serve as an oracle for evaluating cheaper estimators. Do not branch at every token. First identify candidate reasoning boundaries or uncertainty peaks, then allocate larger K where confidence intervals overlap or estimated success is near the decision boundary. ⁵

For active single-turn tree search, prefer adaptive over exhaustive branching. TreeRL’s entropy signal is a reasonable starting selector, but combine entropy with predicted outcome sensitivity. A practical score is

$$\text{priority}(s) = \text{uncertainty}(s) \times \text{estimated value spread}(s) \times \text{remaining-horizon weight}.$$

The first factor finds uncertain decisions, the second avoids branching on stylistic uncertainty, and the third prevents the allocator from spending all compute near the end.

For long-horizon agents, branch at action or agent-step boundaries, not arbitrary tokens. Tree-GRPO is appropriate when shared prefixes contain expensive retrieval or tool calls. BranPO is preferable when the horizon is long and most steps are uninformative, because it searches for contrasting suffixes rather than uniformly expanding all nodes. ⁷³

Use both a local and a global baseline. Local sibling comparisons isolate decisions under a shared prefix, while a prompt-level or inter-tree baseline remains defined when a local branch group is small or uniform. Tree-GRPO’s collapse under intra-tree-only normalization at low branch count is direct evidence for this design. ⁷⁴

Regularize node values by evidence count. Replace raw descendant means at low-count nodes with empirical-Bayes shrinkage,

$$\tilde{V}(s) = \frac{n_s \bar{R}_s + \alpha V_{\text{parent}(s)}}{n_s + \alpha},$$

and propagate uncertainty as well as the mean. Use smaller policy weights for nodes whose posterior intervals are wide. RTMC’s prior ablation suggests this can materially affect training. ⁵⁸

Keep a strict final-answer-only experimental track. Format rewards, validity bonuses, LLM-judge process scores, and environment shaping can be useful in production, but they should be separated from experiments testing whether tree structure alone solves terminal credit assignment. RTMC’s reported setup illustrates why this distinction matters. ⁵⁶

Account for all inference. Report at least total generated tokens, uncached generated tokens, prefix-cache reads, verifier calls, tool calls, environment resets, actor forward/backward FLOPs, wall-clock time, and peak memory. A branch method should not be called sample-efficient merely because it counts several cheap suffixes as one “tree.”

Experimental validation agenda

Core research questions

A useful validation program should answer four questions separately:

1. **Attribution accuracy:** Does the method identify prefixes or actions whose modification causally changes success?
2. **Learning efficiency:** Does better attribution improve final performance per training token, verifier call, wall-clock hour, and optimizer update?
3. **Robustness:** Does the advantage remain useful under verifier noise, rare success, long horizons, and stale rollouts?
4. **Generality:** Does the same estimator transfer across mathematics, code, retrieval agents, and environment-interacting agents without domain-specific shaping?

Proposed benchmark suites

Strict final-reference-answer reasoning suite. Train on MATH or a contamination-controlled mathematical corpus; evaluate on MATH500, GSM-HARD, AMC, AIME, Minerva Math, and OlympiadBench. Add code generation using unit-test terminal rewards on HumanEval+, MBPP+, and LiveCodeBench. The training reward should be exactly binary final-answer or unit-test success, with no step reward. These datasets reflect the regimes already explored by VinePPO, TreeRL, TreeRPO, and TEMPO while permitting common evaluation. ⁷⁵

Multi-hop answer suite. Use HotpotQA, 2WikiMultiHopQA, MuSiQue, and Bamboogle, with exact match and F1 against the final answer. Tool observations may be available to the agent, but only the final answer should enter the reward. These tasks permit comparison of Tree-GRPO and BranPO-style branch construction without relying on environment-specific success shaping. ⁷⁶

Terminal-success agent extension. Use GAIA, ALFWorld, and SWE-bench Verified. This suite relaxes “reference answer” to a terminal environment verifier and should be reported separately. SWE-bench is

particularly useful for state-merging methods, but a strict terminal-only version must remove the intermediate action rewards used in RTMC’s headline experiment. 77

Baselines

The mandatory flat baselines should be supervised fine-tuning without RL, REINFORCE or RLOO, GRPO/ DAPO, PPO with a learned critic, and rejection-sampling self-training. Tree baselines should include TEMPO as the passive-tree method, VinePPO as the high-budget Monte Carlo method, TreeRL or TreeRPO for active single-turn branching, and Tree-GRPO or BranPO for agents. ReST-MCTS* should be included when offline search, a PRM, or synthetic-data generation is permitted. 78

A particularly important baseline is **compute-matched flat resampling**: spend exactly the same generated-token and verifier budget on additional independent complete trajectories rather than branches. Without this control, improvements may result from more terminal samples rather than better prefix-conditioned attribution.

Metrics

Task metrics. Report greedy or temperature-controlled Pass@1, Pass@ k , exact match, F1, unit-test pass rate, and environment success. Include response length and tool calls because some methods improve accuracy by producing longer or more expensive trajectories, while TreeRPO reports shorter outputs. 32

Credit-estimation metrics. For a stratified sample of prefixes, generate 128–256 fresh suffixes to form a high-budget conditional-value oracle. Measure value mean absolute error, Brier score, calibration error, action-ranking accuracy, Kendall correlation, and regret of choosing the action with maximum predicted advantage. VinePPO’s 256-suffix analysis provides a precedent for this evaluation. 79

Causal localization metrics. Select the top-credited and bottom-credited steps. At each step, resample or replace the action while holding the prefix fixed. Measure the absolute change in terminal success probability. A valid fine-grained estimator should show substantially larger intervention effects at top-ranked steps than at random or low-ranked steps.

Efficiency metrics. Plot final performance against generated tokens, newly generated suffix tokens, verifier calls, tool calls, GPU-hours, wall-clock time, and optimizer updates. Report peak memory and the proportion of inference saved through prefix caching.

Tree-quality metrics. Report average branch depth, branching factor, descendants per node, fraction of uniform-reward sibling groups, effective sample size, percentage of singleton nodes, outcome-contrast discovery rate, and the fraction of tokens or steps receiving nonzero local credit.

Training-stability metrics. Report policy KL, entropy, clipping fraction, gradient norm, reward-group standard deviation, collapse frequency, invalid-action rate, and variance across at least five seeds.

Generalization metrics. Measure in-distribution and out-of-distribution accuracy at matched train accuracy, not only at matched optimizer step. VinePPO’s “generalization slope” framing is useful because a method can fit the training verifier quickly while learning less transferable reasoning. 14

Critical ablations

Ablation axis	Values to test	Question answered
Branch selector	Random; token entropy; value uncertainty; gradient sensitivity; outcome-contrastive; learned budget allocator	Is performance due to the tree or to where it branches?
Branch granularity	Token; sentence; semantic reasoning step; tool/agent step; abstract environment state	Which state definition yields the best causal signal per unit cost?
Branch budget	Factors 2, 4, 8; depths 1–5; suffix samples $K = 1, 3, 9, 32$	How quickly do returns diminish with additional counterfactuals?
Value backup	Mean; max; discounted mean; sibling mean; Bayesian shrinkage; confidence lower bound	Is raw descendant averaging statistically adequate?
Advantage composition	Local only; global only; local + global; intra-tree + inter-tree	Does the stable global baseline prevent local-group collapse?
Tree source	Ordinary rollout trie; active on-policy branches; stale branches; teacher/off-policy branches	How much is gained by intervention rather than retrospective structure?
State merging	Exact prefix; hand-designed signature; learned embedding; no merge	What bias-sharing trade-off is introduced by abstraction?
Group filtering	Keep all; prune uniform groups; retain only contrastive branches; inverse-probability correction	Are gains caused by information filtering, and what bias results?
Objective	REINFORCE; PPO clipping; GRPO/DAPO clipping; pairwise DPO; hybrid	Is the estimator or the optimizer responsible for the gain?
Verifier noise	0%, 1%, 5%, 10%, adversarial format errors	Does tree backup amplify terminal-label noise?
Reward regime	Strict terminal only; terminal + format; terminal + validity shaping; learned judge	Which gains remain attributable to final-answer-only credit assignment?
Prefix reuse	Cache disabled; KV reuse only; full environment-state restoration	How much of the claimed scalability is systems rather than statistical?

Recommended flagship experiment

A decisive single-turn study would train the same 7B or 14B base model on the same MATH subset under SFT, PPO, GRPO/DAPO, TEMPO, VinePPO, TreeRL, and TreeRPO. Each method would receive identical totals of newly generated tokens and verifier calls. A separate unconstrained-compute condition would permit each method to use its recommended budget. Every 50 optimizer updates, 500 held-out prefixes would

receive 256-suffix oracle evaluation. The study would report downstream accuracy, conditional-value calibration, causal intervention ranking, KL, output length, and wall-clock cost over five seeds.

A corresponding agent study would compare GRPO, Tree-GRPO, BranPO, passive RTMC, and an actively branched RTMC variant on multi-hop QA. The environment would be deterministic and snapshot-resumable. All methods would use only final-answer correctness, with equal tool calls and newly generated tokens. The key ablation would cross branch strategy—random, entropy, contrastive, and uncertainty-reduction—with state representation—full history, exact action trace, and abstract retrieval state. This factorial design would determine whether gains come primarily from tree-conditioned credit, selective exploration, prefix reuse, or state merging.

Primary-source and reproducibility index

Paper	Venue status	Primary paper	Official or linked code	Data availability	Reproducibility assessment
VinePPO	ICML 2025	PMLR	McGill-NLP/VinePPO	MATH and GSM8K public; splits specified	High
ReST-MCTS*	NeurIPS 2024	NeurIPS	THUDM/ReST-MCTS	Public math/science benchmarks	Medium-high
TreeRL	ACL 2025	ACL Anthology	THUDM/TreeRL	Public math and code benchmarks	High
TreeRPO	arXiv 2025; peer-reviewed venue unspecified	arXiv	Linked repository	Public math benchmarks	Medium
TEMPO/P2T	Findings of ACL 2026	ACL PDF	fatebreaker/tempo	MATH, MedQA, and OOD benchmarks public	High
Tree-GRPO	ICLR 2026	OpenReview	AMAP-ML/Tree-GRPO	Retrieval datasets released; full web-agent release status varies	Medium-high
BranPO	arXiv 2026; venue unspecified	arXiv	YubaoZhao/BranPO	Public QA and agent benchmarks; infrastructure-sensitive	Medium-high

Paper	Venue status	Primary paper	Official or linked code	Data availability	Reproducibility assessment
RTMC	arXiv 2026; venue unspecified	arXiv	Official code unspecified	R2E-Gym and SWE-bench Verified are public; state-signature implementation unspecified	Low-to-medium

Two adjacent directions merit monitoring. TreePO uses segmented tree decoding partly to improve rollout efficiency and tree-based advantage estimation, while the ACL 2026 paper on credit-assignment failure in multi-turn self-reflection proposes a tree-structured GRPO variant with turn-level attribution. They reinforce the trend toward semantically coarser tree nodes, but their principal emphases—serving efficiency and iterative self-reflection—are narrower than the final-reference-answer trajectory-branching focus used for the detailed review above. ⁸⁰

¹ ⁴ ¹⁰ ³⁹ ⁷¹ ⁷⁸ ⁸⁰ <https://arxiv.org/html/2509.18314>

<https://arxiv.org/html/2509.18314>

² ¹⁸ ¹⁹ ²⁰ ²¹ ⁶⁰ ReST-MCTS*: LLM Self-Training via Process Reward Guided Tree Search

<https://arxiv.org/html/2406.03816v2>

³ ⁵ ⁹ ¹² ¹³ ¹⁴ ¹⁵ ⁶⁶ ⁶⁸ ⁶⁹ ⁷⁵ ⁷⁹ <https://arxiv.org/html/2410.01679v2>

<https://arxiv.org/html/2410.01679v2>

⁶ ⁴¹ ⁴² ⁴³ ⁶³ ⁶⁷ ⁷³ ⁷⁴ ⁷⁶ <https://arxiv.org/pdf/2509.21240>

<https://arxiv.org/pdf/2509.21240>

⁷ ³¹ ³² ³³ ³⁴ ³⁵ ³⁶ ³⁷ ⁶² ⁷² <https://arxiv.org/html/2506.05183v2>

<https://arxiv.org/html/2506.05183v2>

⁸ ⁵³ ⁵⁴ ⁵⁵ ⁵⁶ ⁵⁷ ⁵⁸ ⁵⁹ ⁶⁵ <https://arxiv.org/html/2604.11037v1>

<https://arxiv.org/html/2604.11037v1>

¹¹ <https://proceedings.mlr.press/v267/kazemnejad25a.html>

<https://proceedings.mlr.press/v267/kazemnejad25a.html>

¹⁶ VinePPO: Unlocking RL Potential For LLM Reasoning ...

https://github.com/McGill-NLP/VinePPO?utm_source=chatgpt.com

¹⁷ https://proceedings.neurips.cc/paper_files/paper/2024/hash/76ec4dc30e9faaf0e4b6093eaa377218-Abstract-Conference.html

https://proceedings.neurips.cc/paper_files/paper/2024/hash/76ec4dc30e9faaf0e4b6093eaa377218-Abstract-Conference.html

²² ³⁰ <https://aclanthology.org/2025.acl-long.604/>

<https://aclanthology.org/2025.acl-long.604/>

²³ ²⁴ ²⁵ ²⁶ ²⁷ ²⁸ ²⁹ ⁶¹ ⁷⁰ <https://aclanthology.org/2025.acl-long.604.pdf>

<https://aclanthology.org/2025.acl-long.604.pdf>

38 <https://aclanthology.org/2026.findings-acl.524.pdf>
<https://aclanthology.org/2026.findings-acl.524.pdf>

40 <https://openreview.net/forum?id=ZpQwAFhU13>
<https://openreview.net/forum?id=ZpQwAFhU13>

44 45 <https://github.com/AMAP-ML/Tree-GRPO>
<https://github.com/AMAP-ML/Tree-GRPO>

46 47 48 49 50 51 52 64 77 <https://arxiv.org/html/2602.03719v2>
<https://arxiv.org/html/2602.03719v2>